Архитектура и дизайн проекта

1. Общее описание

Проект "Поиск чисел Кита" построен на принципах **объектно-ориентированного программирования (ООП)** и **модульной архитектуры**. Система разделена на независимые компоненты, каждый из которых отвечает за свою конкретную задачу. Такой подход обеспечивает высокую степень переиспользуемости кода, простоту тестирования и возможность масштабирования.

Ключевые особенности архитектуры:

- **Разделение ответственности (SRP)** — каждый класс выполняет одну задачу.
- **Композиция объектов** — классы используют друг друга через вложенные экземпляры.
- **Минимальная связанность** — модули независимы и могут работать автономно.
- **Кроссплатформенность** — код не зависит от операционной системы и использует только стандартную библиотеку Python.

---

2. Принципы проектирования

2.1 Принцип единственной ответственности (Single Responsibility Principle)

Каждый класс в проекте имеет строго определенную зону ответственности:

Класс	Единственная ответственность
`KitChecker`	Проверка одного числа на принадлежность к числам Кита.
`KitFinder`	Перебор диапазона чисел, валидация данных и координация процесса поиска.
`KitBenchmark`	Замер времени выполнения и формирование итогового отчета.
`main.py`	Взаимодействие с пользователем (консольный UI) и обработка исключений.

Преимущества:

- Легкость понимания кода.
- Простота модификации (изменение одного класса не затрагивает другие).
- Удобство тестирования (каждый класс тестируется отдельно).

2.2 Принцип композиции (Composition over Inheritance)

Вместо использования глубокого наследования, проект применяет **композицию** — классы содержат экземпляры других классов как свои поля:

```
KitBenchmark содержит KitFinder
KitFinder содержит KitChecker
```

Почему композиция, а не наследование?

- Более гибкая связь между компонентами.
- Возможность заменять компоненты без изменения иерархии.
- Избегание проблем жесткой связанности, присущих наследованию.
- Лучшая инкапсуляция внутренних деталей.

---

3. Структура взаимодействия классов

Взаимодействие между компонентами построено по иерархическому принципу, где каждый вышестоящий уровень управляет нижестоящим:

```
[ main.py ] (Точка входа / UI)
   |
   └──> [ KitBenchmark ] (Хронометрист)
             |
             └──> [ KitFinder ] (Координатор)
                       |
                       └──> [ KitChecker ] (Ядро алгоритма)
```

Описание связей:

1. **`main.py` → `KitBenchmark`** : Точка входа создает экземпляр бенчмарка, передает ему данные от пользователя и выводит итоговый отчет.
2. **`KitBenchmark` → `KitFinder`** : Бенчмарк содержит искатель как поле (композиция). Он запускает поиск и замеряет время до и после его работы.
3. **`KitFinder` → `KitChecker`** : Искатель содержит проверщик как поле. В цикле он передает каждое число диапазона в `KitChecker` для математической проверки.

---

4. Детальное описание компонентов

4.1 KitChecker — Ядро алгоритма

Ответственность: Проверка одного числа на принадлежность к числам Кита.

Ключевые особенности:

- **Без состояния (stateless)**: Не хранит данные между вызовами.
- **Оптимизация памяти**: Использует алгоритм скользящего окна (sliding window) через сдвиг элементов списка, что предотвращает неограниченный рост памяти.

- **Детерминированность**: Для одного и того же входа всегда возвращает одинаковый результат.

Методы:

- `IsKitNumber(number)` — публичный метод проверки.
- `GetDigits(number)` — приватный метод разбиения числа на список цифр.
- `CumulativeSum(digits, number)` — приватный метод построения последовательности и проверки на совпадение.

4.2 KitFinder — Координатор поиска

Ответственность: Перебор чисел в диапазоне и координация процесса проверки.

Ключевые особенности:

- **Строгая валидация**: Проверяет корректность диапазона до начала вычислений, выбрасывая `ValueError` при ошибках.
- **Итеративный подход**: Последовательно проверяет все числа в заданных границах.

Методы:

- `ValidateRange(start, end)` — приватный/публичный метод валидации границ.
- `FindKitNumbers(start, end)` — основной метод, возвращающий список найденных чисел.

4.3 KitBenchmark — Хронометрист

Ответственность: Замер времени выполнения и формирование отчета.

Ключевые особенности:

- **Высокоточное измерение времени**: Использует `time.perf_counter()`, который не зависит от системных часов ОС и идеален для бенчмарков.
- **Хранение состояния запуска**: Сохраняет параметры последнего запуска (`last_start`, `last_end`) для формирования отчета.

Методы:

- `Run(start, end)` — запуск процесса поиска и замер времени.
- `GetResults()` — возвращает отформатированную строку с результатами.

4.4 Main — Пользовательский интерфейс

Ответственность: Взаимодействие с пользователем.

Ключевые особенности:

- **Безопасный импорт**: Использует конструкцию `if __name__ == "__main__":`, что позволяет импортировать модуль без побочного автозапуска.
- **Обработка исключений**: Перехватывает `ValueError` (как от функции `int()`, так и от `KitFinder`) и выводит понятные сообщения пользователю, не ломая работу программы.

---

5. Поток данных (Data Flow)

Процесс выполнения запроса пользователя выглядит следующим образом:

1. Пользователь вводит границы диапазона `[start, end]`.
2. `main.py` создает объект `KitBenchmark` и вызывает метод `Run()`.
3. `KitBenchmark` засекает время старта (`time.perf_counter()`).
4. `KitBenchmark` вызывает `KitFinder.FindKitNumbers()`.
5. `KitFinder` валидирует диапазон. Если он некорректен — выбрасывается исключение, которое перехватывается в `main.py`.
6. `KitFinder` запускает цикл по всем числам диапазона.
7. Для каждого числа вызывается `KitChecker.IsKitNumber()`.
8. `KitChecker` разбивает число на цифры и строит последовательность, пока сумма не сравняется с числом или не превысит его.
9. Найденные числа собираются в список и возвращаются по цепочке вверх: `KitChecker` → `KitFinder` → `KitBenchmark`.
10. `KitBenchmark` засекает время финиша, вычисляет разницу и сохраняет результаты.
11. `main.py` вызывает `GetResults()` и выводит итог в консоль.

---

6. Паттерны проектирования

6.1 Паттерн "Композиция" (Composition)

Все классы связаны через вложение экземпляров. Это делает систему модульной. Например, если в будущем потребуется изменить алгоритм проверки (например, добавить кэширование), можно будет создать класс `CachedKitChecker` и просто подменить его в `KitFinder` без изменения логики поиска.

6.2 Паттерн "Фасад" (Facade)

Класс `KitBenchmark` выступает в роли упрощенного интерфейса (фасада) для сложной подсистемы. Пользователю (или `main.py`) не нужно знать о существовании `KitFinder` и `KitChecker`, о том, как засекается время или как валидируется диапазон. Ему достаточно вызвать один метод `benchmark.Run()`.

---

7. Обоснование архитектурных решений

7.1 Почему три класса, а не один?

Разделение на `Checker`, `Finder` и `Benchmark` позволяет тестировать каждый компонент изолированно. Именно благодаря этому подходу удалось написать 29 модульных тестов, которые быстро и точно проверяют логику без необходимости запускать весь бенчмарк.

7.2 Почему без абстрактных интерфейсов (ABC)?

В коде не используются абстрактные базовые классы (например, `IBenchmark`). Это сделано намеренно в соответствии с принципом **YAGNI** (You Aren't Gonna Need It — "вам это не понадобится"). Поскольку на данный момент существует только один модуль нагрузки, введение интерфейсов создало бы избыточную абстракцию. При добавлении второго модуля (например, поиска чисел Фибоначчи) интерфейсы могут быть внедрены без переписывания существующей логики.

7.3 Почему реализовано скользящее окно?

В методе `CumulativeSum` вместо постоянного добавления элементов в конец списка (`append`) и суммирования всего списка, используется сдвиг элементов внутри списка фиксированной длины. Это гарантирует, что пространственная сложность алгоритма всегда будет равна $O(N)$, где N — количество цифр в числе, независимо от того, сколько шагов потребовалось для проверки.

---

8. Анализ сложности

8.1 Временная сложность

- **Проверка одного числа:** $O(K \cdot N)$, где K — длина последовательности до совпадения/превышения, N — количество цифр.
- **Поиск в диапазоне:** $O(M \cdot K \cdot N)$, где M — размер диапазона.

8.2 Пространственная сложность

- **KitChecker:** $O(N)$ — благодаря скользящему окну хранится только список из N цифр/чисел.
- **KitFinder / KitBenchmark:** $O(R)$ — хранится только итоговый список найденных чисел Кита, где $R \ll M$.

---

9. Заключение

Архитектура проекта демонстрирует применение современных и чистых принципов проектирования ПО:

- **SRP и Композиция** обеспечили модульность и читаемость.
- **Оптимизация ресурсов** (скользящее окно и `perf_counter`) сделала бенчмарк точным и экономичным.
- **Отсутствие избыточных абстракций** (YAGNI) сохранило код простым и понятным.
- **Разделение UI и бизнес-логики** позволило легко покрыть ядро программы модульными тестами.

Проект полностью соответствует требованиям кроссплатформенности, надежности и готовности к дальнейшему масштабированию.